Integration Test: Flight Search Functionality

The flight search integration test is designed to verify that the various components of the flight search feature are properly integrated correctly and work seamlessly together. The test consists of two main steps:

1. Adding a Flight Search Request (POST Method) which simulates a flight search by sending a POST request with specific parameters for departure city, arrival city, and date (e.g. Toronto to Vancouver on July 6, 2024). The purpose is to conform that the backend processes the flight search request accurately and responds with matching flight records. It checks that the response contains the expected data, including flight details that meet the search criteria.
2. Retrieving Flight Data (GET Method) which involves retrieving flight information using a GET request with the same search parameters. This step help to make sure that the system can correctly fetch flight data from the database, confirming that the records added during the setup are accessible. It verifies that the response includes the correct flight details that validate the functionality of retrieving flight information.

Overall, this integration test confirms that the backend APU, database, and flight search view are properly integrated and functioning as intended. It validates that users can perform a flight search and retrieve accurate results, which is crucial for the flight booking feature.

```
(base) lamchinghei@ChingheideMacBook-Pro backend % python3 manage.py test flights

Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
Response Status Code (POST): 200
Response Data (POST): {'flights': [{'id': 1, 'flight_number': 'NAN9072', 'departure_city': 'Toronto', 'a
rrival_city': 'Vancouver', 'departure_date': '2024-07-06', 'departure_time': '10:00:00', 'arrival_time':
 '15:00:00', 'duration': '5h 0min', 'aircraft': 'Boeing 787', 'price_economy': '389.00', 'price_premium'
: '869.00', 'price_first': '2445.00'}]}
Response Status Code (GET): 200
Response Data (GET): {'flights': [{'id': 1, 'flight_number': 'NAN9072', 'departure_city': 'Toronto', 'ar
rival_city': 'Vancouver', 'departure_date': '2024-07-06', 'departure_time': '10:00:00', 'arrival_time':
'15:00:00', 'duration': '5h 0min', 'aircraft': 'Boeing 787', 'price_economy': '389.00', 'price_premium':
 '869.00', 'price_first': '2445.00'}]}
.
----------------------------------------------------------------------
Ran 1 test in 0.014s

OK
Destroying test database for alias 'default'...
(base) lamchinghei@ChingheideMacBook-Pro backend %
```

Prerequisites

Before running the test, ensure you have the following:

- **Python 3 installed**
- **Django and project dependencies installed**

Getting Started

1. **Clone the Repository** (if you haven't already) and navigate to the Assignment-5 backend folder:
    - git clone https://github.com/mentlesoul/CISC327-Group-22-TA.git
    - cd CISC327-Group-22-TA/Assignment-5/backend
2. **Run the Test Script**: Execute the integration test script by running the following command:
    - python3 manage.py test flights

    This command will:

    - Create a temporary test database
    - Run the integration tests for the flights app
    - Display the results in the terminal
3. **View the Test Output**: The results will indicate whether the test passed or failed and show detailed output about the flight search operations.